

PHASE 1 · WEEKS 1–3 · RAG FOUNDATIONS

# Step-by-Step Project Guide

A build-order walkthrough from empty repo to a working, evaluated, re-ranked RAG assistant.

---

Companion to: Phase 1 — Problem Statement

AI Engineering Masters Program

## How to Use This Guide

Follow the steps in order — each stage depends on the one before it. Don't jump to the UI before retrieval is verified from the command line; don't wire re-ranking before plain retrieval already returns sane chunks. Verify each step with the checklist before moving to the next.

### 1 Pick Your Domain & Freeze Scope

- Choose one option from the Problem Statement (or propose your own and get it approved).
- Collect 1–3 real PDFs for that domain — not lorem-ipsum filler. Real documents expose real chunking problems.
- Write a one-paragraph scope note: what questions your assistant must answer, and what it explicitly won't.
- Checklist: PDFs collected, scope note written, domain confirmed with instructor if it's the open-topic option.

### 2 Scaffold the Repository

- Create the repo with two top-level folders: `backend/` (FastAPI) and `frontend/` (Next.js, or a simpler Streamlit/HTML UI if you're short on time).
- Add `.env.example` for both — never commit real keys.
- Add a Python virtual environment and a `requirements.txt` pinned to the libraries you'll actually use: `fastapi`, `uvicorn`, `langchain`, `langchain-community`, `sentence-transformers`, `faiss-cpu`, `ragas`.
- Checklist: repo pushed to GitHub, backend boots with a placeholder `/health` route returning 200.

### 3 Ingestion & Chunking

- Write a PDF loader (PyPDFLoader or pdfplumber) that extracts text per page, preserving page numbers — you need these for citations later.
- Chunk with a recursive character/token splitter. Start at ~500–800 tokens with ~10–15% overlap; tune after you see retrieval quality.
- Store chunk metadata alongside the text: source filename, page number, chunk index.
- Checklist: uploading a PDF produces a list of chunk objects with correct page numbers, printed to console.

### 4 Embeddings & FAISS Index

- Embed chunks locally with `sentence-transformers/all-MiniLM-L6-v2` — no API key required, and it keeps cost at zero during development.
- Build a FAISS index (`IndexFlatIP` with normalized vectors, or `IndexFlatL2`) and persist it to disk so you don't re-embed on every restart.
- Write a `query_index(question, k)` function that returns the top-k chunks with similarity scores.
- Checklist: same question run twice returns identical top-k results; different questions return visibly different chunks.

### 5 LCEL RAG Chain

- Wire retrieval → prompt template → LLM into a single LCEL chain (or LangGraph if you prefer — either is acceptable as long as the stages are explicit).
- Write a prompt that forces the model to answer only from provided context and to say so explicitly when the answer isn't in the context.

- Return structured output: answer text + list of source chunks used (id, page, snippet).
- Checklist: asking a question answerable from the PDF gives a grounded answer; asking an out-of-scope question gets an honest 'not in this document' response, not a hallucination.

## 6 Query Routing / Decomposition

- Add a lightweight routing step: classify whether the query is a single-fact lookup, a multi-part question, or a summarization request.
- For multi-part questions, decompose into sub-queries, retrieve for each, and merge context before generation.
- This does not need to be elaborate — a single LLM call that returns a JSON list of sub-questions is enough to demonstrate the concept.
- Checklist: a deliberately multi-part test question (e.g. 'What is X and how does it compare to Y?') produces evidence that both parts were retrieved for.

## 7 Re-Ranking

- Add a cross-encoder re-ranker (e.g. ms-marco-MiniLM-L-6-v2) that re-scores the top-N chunks retrieved by FAISS before they reach the LLM.
- Log both the pre-rerank and post-rerank ordering somewhere you can screenshot for the README.
- Checklist: at least one test query exists where re-ranking visibly changes the top result's rank — capture it as a before/after example.

## 8 RAGAS Evaluation

- Build a small labeled eval set: 8–15 question/expected-answer pairs covering your document.
- Run RAGAS metrics — faithfulness, answer relevancy, context precision — after each query, or in a batch eval script.
- Log results to a JSONL file (timestamp, question, answer preview, metrics) — mirror the reference project's log format.
- Checklist: eval script runs top to bottom without manual intervention and produces a metrics table.

## 9 Frontend & Source Attribution UI

- Build the minimum UI needed to demo the pipeline: upload box, chat/question box, answer display, and an expandable 'sources' panel showing cited chunks and page numbers.
- Voice input/output, topic sidebars, and suggestion chips (as in the reference project) are optional polish — not required for a passing submission.
- Checklist: a fresh reviewer can upload a PDF and get a cited answer within 60 seconds of opening the app, with no setup help from you.

## 10 Write the README & Deploy

- Document setup (local + hosted), architecture, tech stack table, and API reference — see the Problem Statement for the required depth.
- Deploy per the Vibe Coding Helper Doc, then re-test the deployed link end to end (not just localhost) before submitting.
- Checklist: a stranger with only your README and repo link can get the app running without asking you a single question.

